
Cours n° 1

INTRODUCTION

I LES BASES**C'est un quoi un ordinateur?**

- Pour fonctionner, ils ont besoin d'une source d'énergie, en l'occurrence d'électricité.
- Ils reçoivent des informations de la part de l'utilisateur, par l'intermédiaire d'un clavier, d'une souris, d'un écran tactile, d'un réseau.
- Ils émettent des informations, par l'intermédiaire de l'écran, du haut-parleurs ou du réseau.

Oui mais :

Une automobile aussi, une thermostat d'ambiance également. Et on a pas trop envie d'appeler ça des ordinateurs.

Historique :

Depuis la création des premières villes il y a 5000 ans en Mésopotamie, il est devenu nécessaire de traiter des informations de manière automatisée, ce que les mathématiciens ont essayé de faire tout au long de l'Histoire. La Pascacine, de Blaise Pascal, en 1642 permettait d'effectuer les quatre opérations usuelles sur les entiers. Au XIX^e siècle, des machines sont construites pour calculer et imprimer des tables de logarithme. Ce pose alors la question : peut-on tout calculer avec une machine? En 1928 David Hilbert se demande si il existe une machine capable de décider si une proposition mathématique est vraie ou fausse.

Alonzo Church et Alan Turing répondent indépendamment non en 1936 et 1937.

L'article de Turing propose un modèle de machine (qu'on appelle machine de Turing) qui possède les caractéristiques suivantes :

- Elle possède un ruban infini sur lequel on a disposé des données. Elle peut lire les données, les traiter et en écrire d'autres. La machine peut également s'arrêter au bout d'un certain temps.
- Pour tout procédé calculable par un algorithme, il existe une machine de Turing capable de le calculer.
- Il existe une machine de Turing universelle, c'est à dire une machine capable de simuler toutes les autres : on décrit la machine que l'on veut simuler sur le ruban.
- Il démontre également que cette machine n'est pas capable de résoudre certains problèmes, comme de décider si une proposition mathématique est vraie, ou simplement si une machine s'arrêtera.

Grâce a cet article révolutionnaire, on définit un ordinateur : *un ordinateur est la réalisation concrète d'une machine de Turing universelle, c'est à dire une machine traitant des informations et capable en principe de prendre comme donnée d'entrée n'importe quel algorithme et de l'exécuter.*

Et donc :

Ordinateur, tablettes, smartphones sont des ordinateurs.

UN GPS, un lecteur DVD et tout dans ce genre qui contient un tant soit peu d'électronique n'est pas un ordinateur : ils sont incapables d'exécuter des algorithmes arbitraires.

La "machine" dans ces appareils est appelée un microcontrôleur, et on parle de systèmes embarqués.

II LANGAGE

Un ordinateur permet donc de créer et d'exécuter des programmes. C'est à la base une chose ardue : on ne dispose que de quelques instructions :

- ▷ LOAD : charge une valeur mémoire vers le registre
- ▷ STORE : l'opération inverse
- ▷ ADD : somme de deux registres, placé dans un troisième
- ▷ BNE : Branch if Not Equal, pour les tests

C'est à dire que pour faire une somme on fait :

```
LOAD R0, B
LOAD R1, C
ADD R2, R0, R1
STORE R2, A
```

On appelle cela l’assembleur.

Et comme ce n’est pas pratique : on conçoit des logiciels pour écrire des programmes, dans certains langages, mais qui au final vont aller charger des informations dans des registres et travailler avec le processeur. C’est le langage qui traduira le code en assembleur.

- Un logiciel permettant d’écrire des programmes est appelé un *environnement de développement intégré* ou IDE, il permet :
- ▷ d’écrire un programme
 - ▷ d’exécuter les programmes
 - ▷ de corriger (débuguer) ces programmes
 - ▷ de consulter une documentation

Un langage de programmation est une notation conventionnelle pour écrire des algorithmes. Il en existe de grands nombres ayant des utilités diverses et variées :

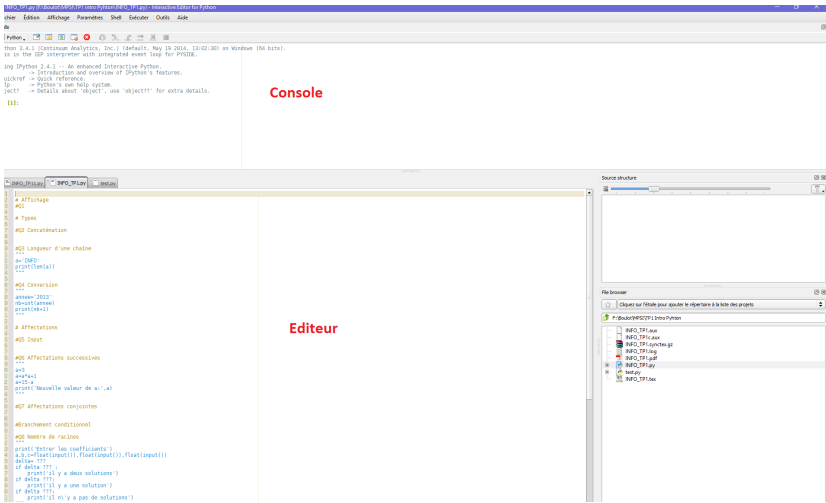
- ▷ les anciens : BASIC, Fortran
- ▷ les pédagogiques : pseudo code de algobox, Pascal
- ▷ exotiques : Malboge
- ▷ scientifique : langage Mapple, Matlab
- ▷ les poids lourds : Java, C, C++, Php,... et notre nouvel ami : Python

Ils sont très variés et sont classés par différents aspects dont on ne se préoccupera pas. (Ce n’est pas difficile mais il y a beaucoup de notions).

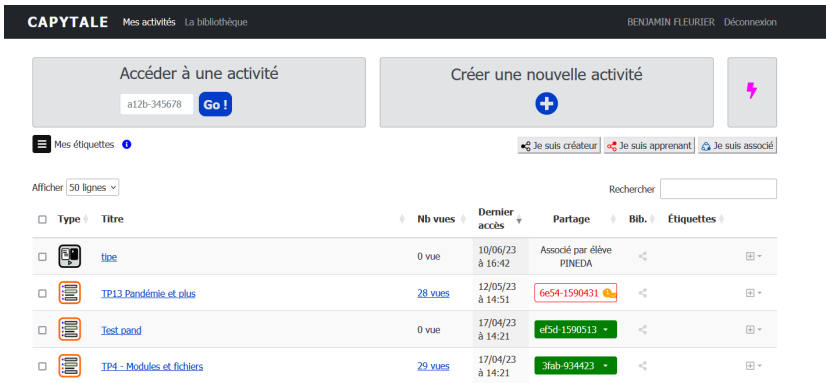
III PYHTON, PYZO ET CAPYTALE

Python est un lange de programmation que vous avez surement déjà utilisé au lycée, mais dans le doute on va reprendre un peu tout.

Pyzo : un logiciel pour coder proprement en Python .



Capytale : une plateforme en ligne facilitant les échanges, parfait pour les TP et pour les DM.



IV INSTRUCTIONS

1 EXPRESSION ET AFFECTATION

- Une *expression* est utilisée pour calculer une valeur ou pour appeler une procédure (une fonction qui renvoie la valeur None). Une expression est composée de noms (ou identifiants), de nombres, de chaînes de caractères, d'éléments séparés par des signes de ponctuation, entourés de parenthèses, de crochets, d'accolades et d'opérateurs. Une expression a une valeur.
- Une *affectation* est utilisée pour lier un nom à une valeur ou pour modifier un élément d'un objet mutable comme une liste. L'instruction affecter est représentée par le signe =. La syntaxe est : `nom = expression` ou `nom[i] = expression` pour les tableaux.

En Python, une expression et une affectation sont deux instructions particulières.

2 TYPE

Le résultat d'une expression ou une variable possède un **type** donné.

Nous utiliserons principalement les types de variables suivants :

- Les entiers, integer, int
- Les booléens, bool, True ou False
- Les flottants, floats : représentation des valeurs approchées des nombres réels
- Les tableaux et listes, list : composés de plusieurs valeurs, modifiables
- Les chaînes de caractères, str : pour représenter les ... caractères
- Les n-uplet, tuples : des listes non modifiables

En Python on peut obtenir le type d'une valeur via la commande type

```
>>> type(42)
<class 'int'>
>>> type('bill')
<class 'str'>
```

3 INSTRUCTIONS

Une *instruction* est un morceau de code minimal qui produit un effet. Une instruction est exécutée par une machine.

Une *instruction simple* peut s'écrire sur une seule ligne. On peut en écrire plusieurs séparées par des points-virgules. Outre l'expression et l'affectation, quelques instructions simples sont :

- *affirmer* avec `assert`, qui est suivi d'une expression, (précisions plus tard)
- *renvoyer* avec `return`, qui est suivi d'une expression et s'emploie uniquement dans une fonction, (une expression peut être un n-uplet écrit sans parenthèse)
- *arrêter* avec `break`, mot isolé qui permet d'arrêter une boucle
- *importer* avec `import`, suivi d'un nom de module, (ou de fonction, de constante, appartenant à un module qui est précisé).

Une *instruction composée* est une instruction sur une ligne terminée par : suivie d'une ou plusieurs instructions indentées : on dispose par exemple de `if` (qui peut être suivi de `elif`, de `else`) pour exécuter des instructions selon une condition, de `for` et de `while` pour exécuter des instructions de manière répétée, de `def` pour définir une fonction.

Exceptée les expressions, une instruction n'a pas de valeur.

Le **branchement conditionnel** fameux `si alors (sinon)`. La syntaxe est très simple en Python :

```
1 if condition :
2     instruction1
3 else :
4     instruction2
```

Sachant que le bloc `else` est optionnel. La condition est un booléen : une expression dont le résultat est True ou False.

EXEMPLE 1.

```
1 a = 15
2 b = 10
3 c = 7
4 if a + b == c :
5     print('La somme est juste')
6 else :
7     print('La somme est fausse')
```

V FONCTIONS, MODULES,

1 FONCTION

DÉFINITION 1

Une fonction est une partie du code, relativement indépendante du reste du programme, qui peut être appelée plusieurs fois pour produire un effet, dont le résultat dépend des valeurs données à cette fonction, appelées **arguments**.

EXEMPLE 2.

```
1 def somme(a,b,c) :  
2     return( a+b+c)
```

```
1 def calcul(a,b,c) :  
2     if a == 0 :  
3         return(0)  
4     else :  
5         return(b*c/a)
```

2 MODULES

Une bonne partie des fonctions "basiques" existent en python. `abs(x)` donne la valeur absolue d'un entier ou d'un flottant.

En général dans l'écriture d'un programme, il n'est pas question de réécrire ce qui existe déjà : question de temps et de productivité. Un programme en Python commence souvent par des lignes contenant le mot `import` afin d'utiliser des modules et des bibliothèques.

Un *module* peut être un fichier unique (extension `py`). Il contient des variables, des fonctions, que l'on pourra utiliser dans le programme une fois importé. Un *package* est un ensemble de dossiers et de sous dossiers et une *bibliothèque* est constitué des modules et de packages.

La bibliothèque standard de Python contient un grand nombre d'outils, mais quelques packages supplémentaires seront nécessaires dans l'année. On importe une bibliothèque/un module via la commande `import nom_du_module`, et les fonctions/variables sont utilisées via `nom_du_module.nom_du_truc`

Les principales bibliothèques que nous allons utiliser sont :

- Math : qui contient les fonctions usuelles et compagnie
- Random : pour tout ce qui est hasard
- Matplotlib : pour faire des graphiques, surtout en physique
- Numpy : des outils d'analyse numérique, surtout en physique

VI EXERCICES

EXERCICE 1. Quel est le type adapté pour représenter chacune des données suivantes :

- 1) la taille d'un individu en mètres
- 2) l'âge d'un individu en années
- 3) le nom d'une personne
- 4) les coordonnées d'un pixel
- 5) un numéro de téléphone
- 6) un code pin
- 7) une adresse postale
- 8) une date et une heure

EXERCICE 2. Quel est le résultat de l'appel `what(5,1)` ?

```
1 def what(a,b):  
2     if a+b > 6 :  
3         return 3  
4     else :  
5         return 2*a+3*b
```